

Experiment Report: MNIST Classification Using MLP

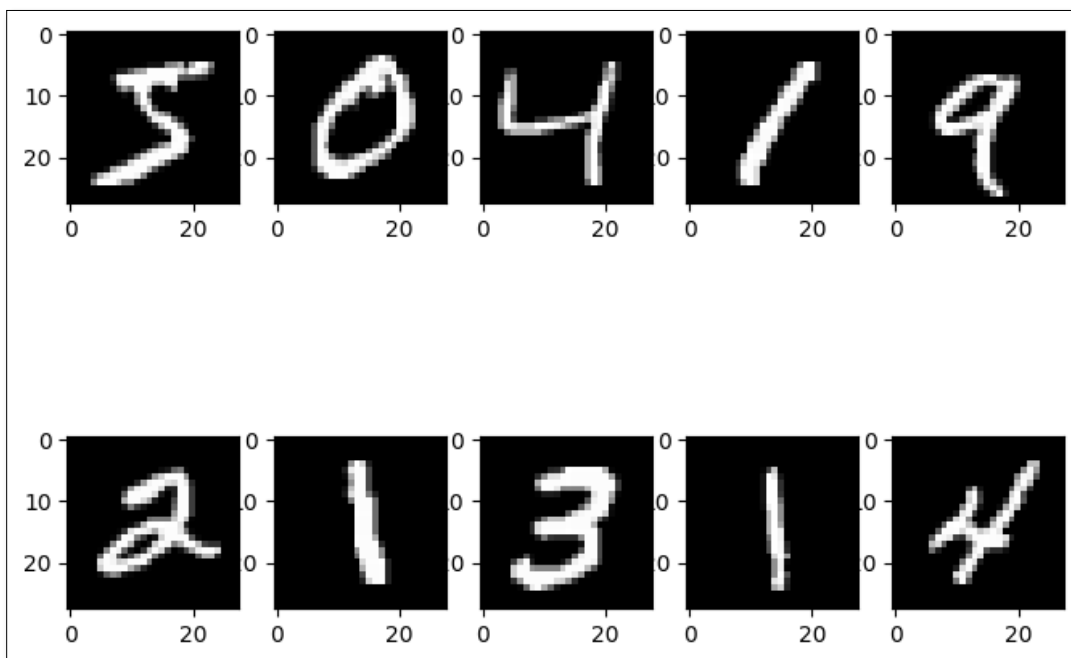
1. Objective

The goal of this experiment was to apply and compare different configurations of **Multilayer Perceptron (MLP)** classifiers on the MNIST handwritten digit dataset to study how varying the **hidden layer structure** and **training iterations** affects classification accuracy, loss convergence, and overall performance.

2. Dataset

- **Dataset:** MNIST (Modified National Institute of Standards and Technology)
- **Loaded from:** `fetch_openml('mnist_784')`
- **Shape of data:** (70,000, 784)
- **Shape of labels:** (70,000,)
- **Number of classes:** 10 (digits 0–9)
- **Each image:** 28×28 grayscale flattened into 784 features.

Example of dataset visualization:



3. Experimental Setup

The dataset was divided into training and test subsets (80/20).

All experiments used the same preprocessing and evaluation metrics for a fair comparison.

Common parameters for both experiments:

Parameter	Value
Activation Function	ReLU
Solver	Adam
Learning Rate	Constant
Learning Rate Init	0.01
Batch Size	128
Random State	42
Dataset	MNIST (784 features, 10 classes)

4. Experiment 1

Model Configuration:

MLPClassifier(hidden_layer_sizes=(256, 128),

activation='relu',

solver='adam',

learning_rate='constant',

learning_rate_init=0.01,

batch_size=128,

max_iter=100,

random_state=42)

Key Parameters:

- Two hidden layers: **256 neurons** and **128 neurons**
- **max_iter = 100**

Results:

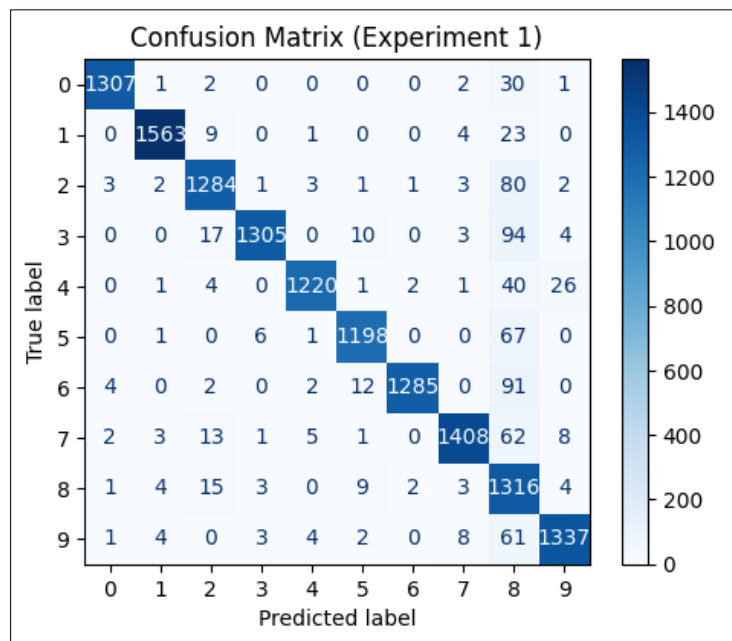
- **Training Time:** 70.77 seconds
- **Accuracy: 0.9445 (94.45%)**

Classification Report (summary):

Metric	Precision	Recall	F1-Score
Macro Avg	0.95	0.94	0.95
Weighted Avg	0.95	0.94	0.95

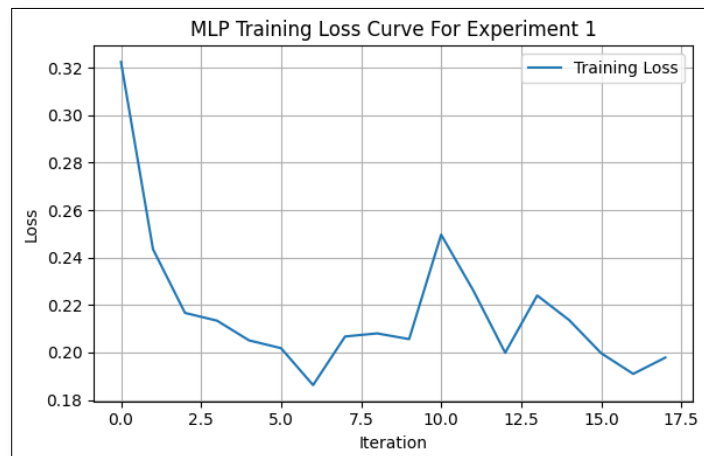
Confusion Matrix:

- Most digits were classified correctly.
- Some confusion occurred between **digit 5 and 8**, and between **3 and 5**.



Training Loss Curve:

- Loss started around 0.32 and decreased gradually to ~0.18.
- Minor fluctuations indicate occasional learning instability due to a relatively large learning rate (0.01).



5. Experiment 2

Model Configuration:

MLPClassifier(hidden_layer_sizes=(10, 10),

activation='relu',

solver='adam',

learning_rate='constant',

learning_rate_init=0.01,

batch_size=128,

max_iter=200,

random_state=42)

Key Differences from Experiment 1:

- Smaller hidden layers: only **10 neurons each**

- **max_iter** increased to **200**

Results:

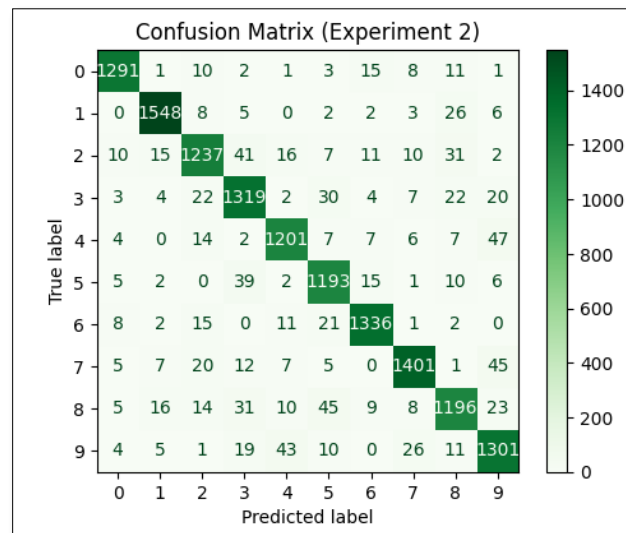
- **Training Time:** 43.84 seconds
- **Accuracy:** 0.9302 (93.02%)

Classification Report (summary):

Metric	Precision	Recall	F1-Score
Macro Avg	0.93	0.93	0.93
Weighted Avg	0.93	0.93	0.93

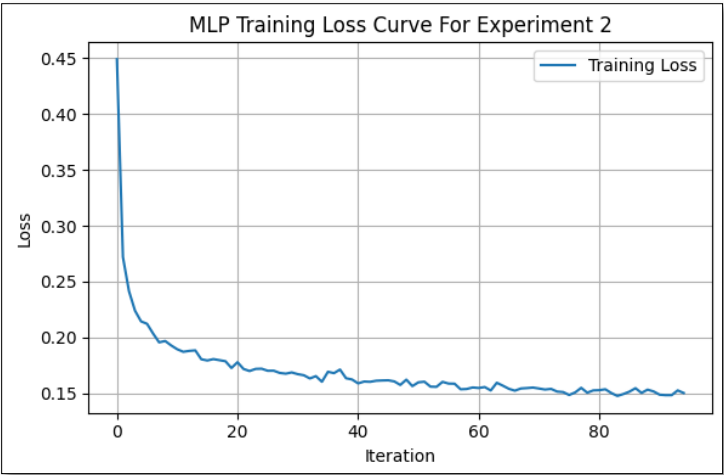
Confusion Matrix:

- Increased misclassifications compared to Experiment 1.
- Overlap between digits with similar shapes (like 4–9 and 3–5).



Training Loss Curve:

- Loss decreased smoothly from 0.45 to 0.15 with no major spikes.
- Indicates **stable convergence** and less overfitting due to smaller network size.



6. Comparative Analysis

Metric	Experiment 1	Experiment 2
Hidden Layers	(256, 128)	(10, 10)
max_iter	100	200
Training Time	70.8 sec	43.8 sec
Accuracy	94.45%	93.02%
Macro Avg F1	0.95	0.93
Convergence	Slightly unstable	Smooth
Overfitting Risk	Moderate	Low

Observation:

- Increasing the number of neurons significantly improved accuracy (Exp-1 > Exp-2).
- Increasing max_iter helped convergence but could not compensate for the smaller network capacity.
- The confusion matrices confirmed that the deeper model (Exp-1) captures more complex digit patterns.

- However, Experiment 2 trained faster and exhibited a smoother loss curve, showing better generalization and stability for smaller models.

7. Conclusion and Discussion

- The **architecture depth and neuron count** play a crucial role in feature extraction and learning capacity.
- **Experiment 1** achieved higher accuracy due to its **larger hidden layers**, enabling it to learn more abstract digit patterns.
- **Experiment 2** demonstrated faster convergence and stability, but limited neuron capacity reduced its ability to differentiate between similar digits.
- Optimal performance may be achieved by **balancing model size and training iterations**, e.g., (128, 64) with max_iter=150.
- In future experiments, adding **regularization** (e.g., dropout or L2 penalty) could further prevent overfitting.

Submitted By:

Md. Shadman Shakib

15th October 2025